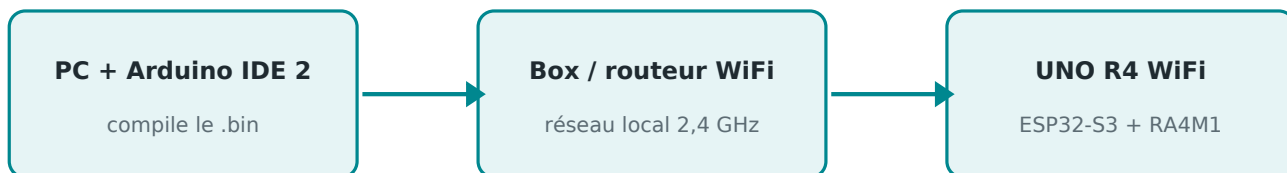


MANUEL D'UTILISATION

Téléversement sans fil (OTA) pour Arduino UNO R4 WiFi

Fonctions `activerOTA()`, `gererOTA()` et `modeOTA()` — intégration dans n'importe quel programme

Mot de passe OTA demandé par l'IDE



HTTP vers le port TCP 65280 • découverte automatique par mDNS (UDP 5353)

Fichier associé : `OTA_R4_WiFi.ino` • Bibliothèque : `ArduinoOTA` (J. Andrassy) • Version du manuel : 1.0 — octobre 2026

Sommaire

1. Principe de fonctionnement	3
2. Matériel et logiciels nécessaires	4
3. Installation (à faire une seule fois)	4
4. Intégrer l'OTA dans un programme quelconque	5
5. Référence des fonctions	6
6. Paramètres de réglage	6
7. Procédure de téléversement	6
8. Mode OTA dédié et sécurité de l'installation	7
9. Dépannage	9
10. Alternative officielle : mise à jour par téléchargement	9
11. Récapitulatif rapide	10
Annexe — Listing complet de OTA_R4_WiFi.ino	11

1. Principe de fonctionnement

L'OTA (*Over The Air*) permet d'envoyer un nouveau programme à l'UNO R4 WiFi par le réseau WiFi, exactement comme avec le bouton « Téléverser » de l'IDE, mais sans câble USB. C'est très pratique pour une carte installée dans un boîtier, un robot ou un endroit difficile d'accès.

L'UNO R4 WiFi contient deux microcontrôleurs : le **Renesas RA4M1** qui exécute votre programme, et un **ESP32-S3** qui gère le WiFi (bibliothèque `WiFi53`). L'IDE Arduino ne sait pas, à lui seul, téléverser par WiFi sur cette carte : on utilise donc la bibliothèque **ArduinoOTA** de Juraj Andrassy, qui prend officiellement en charge l'UNO R4 WiFi.

Déroulement d'une mise à jour :

- Votre programme contient le service OTA, qui écoute sur le port TCP **65280** et s'annonce sur le réseau (mDNS).
- L'IDE détecte la carte et l'affiche comme **port réseau** dans *Outils* > *Port*.
- Au téléversement, l'IDE compile, demande le mot de passe OTA et envoie le fichier `.bin`.
- La carte stocke le nouveau programme dans la seconde moitié de sa mémoire flash, le vérifie, le recopie à la place de l'ancien puis redémarre.



Flash 256 Ko du RA4M1 : le programme ne doit pas dépasser la moitié (≈ 128 Ko)

Conséquence importante — comme le nouveau programme est d'abord stocké dans la moitié libre de la flash, la taille d'un programme compatible OTA est limitée à environ **128 Ko** (l'IDE affiche « Le croquis utilise ... (xx %) » : restez sous ~ 48 %).

Règle d'or — **chaque** programme envoyé par OTA doit lui-même contenir les blocs OTA. Si vous envoyez un programme qui ne les contient pas, il fonctionnera, mais la carte ne sera plus joignable par WiFi : il faudra revenir au câble USB.

2. Matériel et logiciels nécessaires

Élément	Détail
Carte	Arduino UNO R4 WiFi (l'UNO R4 Minima n'a pas de WiFi)
Réseau	WiFi 2,4 GHz (l'ESP32-S3 ne gère pas le 5 GHz). PC et carte sur le même réseau local
IDE	Arduino IDE 2.x (recommandé) ou 1.8.x
Paquet de cartes	« Arduino UNO R4 Boards » (dossier <code>renesas_uno</code>), via le Gestionnaire de cartes
Bibliothèque	« ArduinoOTA » de Juraj Andrassy , version 1.1.0 ou plus récente (Gestionnaire de bibliothèques)
Outil d'envoi	Programme <code>arduinoOTA</code> (installé avec le paquet « Arduino AVR Boards », présent par défaut dans l'IDE)
Firmware WiFi	Firmware ESP32-S3 à jour conseillé : <i>Outils > Firmware Updater</i> dans l'IDE
Câble USB-C	Uniquement pour le tout premier téléversement (et en secours)

3. Installation (à faire une seule fois)

Étape 1 — Installer la bibliothèque ArduinoOTA

Dans l'IDE : *Croquis > Inclure une bibliothèque > Gérer les bibliothèques*, rechercher **ArduinoOTA** et installer celle de **Juraj Andrassy** (ne pas confondre avec d'autres bibliothèques au nom proche).

Étape 2 — Copier le fichier `platform.local.txt`

Ce fichier indique à l'IDE comment envoyer un programme par le réseau à une carte Renesas. Il se trouve dans la bibliothèque, sous-dossier `extras/renesas` :

```
Windows : Documents\Arduino\libraries\ArduinoOTA\extras\renesas\platform.local.txt
macOS   : ~/Documents/Arduino/libraries/ArduinoOTA/extras/renesas/platform.local.txt
Linux   : ~/Arduino/libraries/ArduinoOTA/extras/renesas/platform.local.txt
```

Copiez-le **à côté du fichier `platform.txt`** du paquet de cartes UNO R4 (remplacez `<version>` par le numéro du dossier présent, par ex. 1.5.1) :

```
Windows : %LOCALAPPDATA%\Arduino15\packages\arduino\hardware\renesas_uno\<version>\
macOS   : ~/Library/Arduino15/packages/arduino/hardware/renesas_uno/<version>/
Linux   : ~/.arduino15/packages/arduino/hardware/renesas_uno/<version>/
```

À savoir — à chaque mise à jour du paquet « Arduino UNO R4 Boards », un nouveau dossier `<version>` est créé : il faut **recopier `platform.local.txt`** dedans, sinon le téléversement réseau ne fonctionne plus.

Étape 3 — Vérifier la présence de l'outil `arduinoOTA`

Contrôlez qu'il existe un dossier `packages/arduino/tools/arduinoOTA/` dans le même dossier `Arduino15`. S'il manque, installez « Arduino AVR Boards » depuis le Gestionnaire de cartes.

Étape 4 — Mettre à jour le firmware WiFi (conseillé)

Carte branchée en USB : *Outils > Firmware Updater*, sélectionner l'UNO R4 WiFi, puis *Install*.

Étape 5 — Redémarrer l'IDE

Fermez complètement puis rouvrez l'IDE pour qu'il prenne en compte `platform.local.txt`.

4. Intégrer l'OTA dans un programme quelconque

Le fichier `OTA_R4_WiFi.ino` est découpé en **4 blocs** balisés par des commentaires `#####`. Recopiez-les dans votre programme aux emplacements indiqués.

Bloc 1 — En-tête et réglages (tout en haut du programme)

```
#include <WiFiS3.h>          // Pilote WiFi de l'UNO R4 WiFi – TOUJOURS avant ArduinoOTA.h
#include <ArduinoOTA.h>      // Bibliothèque ArduinoOTA (J. Andrassy)

const char OTA_SSID[] = "NomDeMonReseau"; // Réseau WiFi 2,4 GHz
const char OTA_WIFIPW[] = "MotDePasseWiFi";
const char OTA_NOM[] = "UNO-R4-OTA"; // Nom affiché dans l'IDE
const char OTA_MDP[] = "password"; // Mot de passe OTA -> à changer !

#define OTA_LED_PIN 13 // LED témoin, -1 pour désactiver
#define OTA_WIFI_TIMEOUT 20000UL // ms
#define OTA_RECO_PERIODE 30000UL // ms
#define OTA_DEBUG 1 // messages série
/* + variables internes et macros OTA_LOG (voir fichier .ino) */
```

Ordre des #include — `WiFiS3.h` doit impérativement être inclus **avant** `ArduinoOTA.h`, sinon l'erreur « 'ArduinoOTA' was not declared in this scope » apparaît.

Bloc 2 — Les fonctions (avant `setup()`)

Copiez intégralement le bloc 2 : `otaLed()`, `connecterWiFi()`, les deux fonctions de rappel, `activerOTA()`, `gererOTA()` et `modeOTA()`. Vous n'avez rien à y modifier, sauf éventuellement la fonction `otaAuDebut()` (voir § 8).

Blocs 3 et 4 — Une ligne dans `setup()`, une ligne dans `loop()`

```
void setup() {
  Serial.begin(115200);
  activerOTA(); // <-- BLOC 3 : passe la carte en mode OTA
  // ... votre setup() ...
}

void loop() {
  gererOTA(); // <-- BLOC 4 : écoute l'IDE + reconnexion WiFi automatique
  // ... votre loop() ...
}
```

delay() et boucles bloquantes — l'OTA n'est écouté que lorsque `gererOTA()` est appelé. Un `delay(10000)` dans `loop()` fera attendre l'IDE jusqu'à 10 s, voire échouer. Préférez une programmation non bloquante avec `millis()`, ou appelez `gererOTA()` à l'intérieur de vos longues boucles.

5. Référence des fonctions

Fonction	Rôle	Bloquante ?
activerOTA()	Passe la carte en mode OTA : connecte le WiFi si nécessaire, vérifie le firmware, déclare les fonctions de rappel et démarre le service ArduinoOTA. Sans effet si déjà actif.	Oui, pendant la connexion WiFi (≤ 20 s)
gererOTA()	À appeler à chaque tour de <code>loop()</code> . Traite les demandes de l'IDE (<code>ArduinoOTA.poll()</code>), détecte la perte du WiFi ou un changement d'IP et relance le service automatiquement.	Non (sauf reconnexion, max. toutes les 30 s)
modeOTA(durée)	Mode OTA dédié : suspend le programme, fait clignoter la LED et attend uniquement un téléversement pendant <i>durée</i> ms (0 = infini). Puis reprend le programme.	Oui (volontairement)
connecterWiFi(t)	Connexion au réseau avec délai max <i>t</i> ms ; attend l'attribution de l'adresse IP. Renvoie <code>true</code> si OK.	Oui
otaAuDebut()	Rappel exécuté au début d'un téléversement : allume la LED. Emplacement prévu pour mettre en sécurité moteurs et actionneurs.	—
otaAvantApplication()	Rappel exécuté quand le programme est reçu, juste avant l'écriture et le redémarrage.	—
otaLed(état)	Commande de la LED témoin (ignorée si <code>OTA_LED_PIN = -1</code>).	Non

6. Paramètres de réglage

Paramètre	Défaut	Description
OTA_SSID / OTA_WIFIPW	—	Identifiants du réseau WiFi (2,4 GHz).
OTA_NOM	UNO-R4-OTA	Nom sous lequel la carte apparaît dans <i>Outils</i> > <i>Port</i> . Un nom différent par carte si vous en avez plusieurs. Pas d'espace ni d'accent.
OTA_MDP	password	Mot de passe demandé par l'IDE à chaque envoi. À changer.
OTA_LED_PIN	13	Broche de la LED témoin « L ». Mettre -1 si D13 sert à autre chose (ex. bus SPI).
OTA_WIFI_TIMEOUT	20000	Durée max (ms) d'une tentative de connexion WiFi.
OTA_RECO_PERIODE	30000	Intervalle (ms) entre deux tentatives de reconnexion si le WiFi est perdu.
OTA_DEBUG	1	1 : messages [OTA] sur le moniteur série ; 0 : aucun message.

7. Procédure de téléversement

7.1 Premier téléversement — par câble USB

- Ouvrez `OTA_R4_WiFi.ino` (ou votre programme avec les 4 blocs) et renseignez SSID, mot de passe WiFi, nom et mot de passe OTA.
- Branchez la carte en USB, sélectionnez la carte **Arduino UNO R4 WiFi** et son port série (COMx ou /dev/tty...).
- Téléversez normalement, puis ouvrez le moniteur série à **115200 bauds**.
- Notez la ligne : `[OTA] Pret ! Port reseau : UNO-R4-OTA at 192.168.1.42`

7.2 Téléversements suivants — par WiFi

- La carte peut être débranchée de l'USB et alimentée ailleurs.
- Dans *Outils* > *Port*, section **Ports réseau**, choisissez UN0-R4-OTA at 192.168.1.42 (l'apparition peut prendre jusqu'à une minute).
- Cliquez sur **Téléverser**. L'IDE demande le **mot de passe** : saisissez la valeur de OTA_MDP.
- La console affiche successivement *Connecting to board... done*, *Uploading sketch... done*, *Flashing sketch... done*. La carte redémarre seule sur le nouveau programme.

```
Connecting to board ... done
Uploading sketch ... done
Flashing sketch ... done
```

Vérifier la mise à jour — modifiez par exemple le texte « (version 1) » de l'exemple en « (version 2) », téléversez par WiFi et observez le changement — via le moniteur série si la carte est encore en USB.

7.3 Méthode de secours n°1 — ligne de commande

Si le port réseau n'apparaît pas (mDNS bloqué par le pare-feu ou le routeur), exportez le binaire (*Croquis* > *Exporter les binaires compilés*) puis lancez l'outil `arduinoOTA` qui se trouve dans `Arduino15/packages/arduino/tools/arduinoOTA/<version>/bin/` :

```
arduinoOTA -address 192.168.1.42 -port 65280 -username arduino -password password
            -sketch MonProgramme.ino.bin -upload /sketch -b
```

(Tout sur une seule ligne ; remplacez l'adresse IP, le mot de passe et le nom du fichier.)

7.4 Méthode de secours n°2 — « programmeur » réseau

Ajoutez à la fin du fichier `programmers.txt` du dossier `renesas_uno/<version>` :

```
arduinoOTA42.name=Arduino OTA (192.168.1.42)
arduinoOTA42.program.tool=arduinoOTA
arduinoOTA42.program.tool.default=arduinoOTA
arduinoOTA42.ip=192.168.1.42
```

Redémarrez l'IDE, choisissez ce programmeur dans *Outils* > *Programmeur*, puis *Croquis* > *Téléverser avec un programmeur*. Dans ce mode, le mot de passe est fixé à `password` dans `platform.local.txt` : modifiez-le dans ce fichier si vous avez changé OTA_MDP. **Fermez le moniteur série** pendant l'envoi.

Conseil — réservez une **adresse IP fixe** (bail DHCP statique) à la carte dans votre box : l'adresse ne changera plus et les méthodes de secours resteront valables.

8. Mode OTA dédié et sécurité de l'installation

8.1 OTA permanent ou OTA à la demande ?

Mode	Mise en œuvre	Avantages / inconvénients
Permanent (recommandé)	<code>activerOTA()</code> dans <code>setup()</code> + <code>gererOTA()</code> dans <code>loop()</code>	Mise à jour possible à tout moment. Le WiFi reste actif en permanence.
À la demande	<code>modeOTA(120000UL)</code> déclenché par un bouton, un interrupteur, une commande série...	OTA ouvert seulement quand on le décide. Nécessite une action physique ou un signal.

Exemple : maintenir un bouton (entre D2 et GND) appuyé à la mise sous tension pour ouvrir 2 minutes de mode OTA :

```
#define BOUTON_OTA 2

void setup() {
  Serial.begin(115200);
  pinMode(BOUTON_OTA, INPUT_PULLUP);
  if (digitalRead(BOUTON_OTA) == LOW) { // bouton appuyé au démarrage ?
    modeOTA(120000UL); // attente d'un téléversement pendant 2 min
  }
  // ... votre setup() ...
}

void loop() {
  // pas de gererOTA() ici : l'OTA n'est disponible qu'au démarrage
  // ... votre loop() ...
}
```

Autre exemple : entrer en mode OTA en tapant la lettre « o » dans le moniteur série :

```
if (Serial.available() && Serial.read() == 'o') modeOTA(0); // 0 = attente infinie
```

8.2 Mettre les actionneurs en sécurité

Pendant la réception (quelques secondes) votre programme ne tourne plus normalement, puis la carte redémarre. Si elle pilote des moteurs, un chauffage, un relais..., complétez `otaAuDebut()` :

```
void otaAuDebut() {
  OTA_LOGLN(F("[OTA] Reception d'un nouveau programme..."));
  otaLed(true);
  analogWrite(PIN_MOTEUR, 0); // arrêt moteur
  digitalWrite(PIN_RELAIS, LOW); // relais au repos
}
```

8.3 Sécurité réseau

- Changez le mot de passe par défaut password.
- Le mot de passe et le programme circulent **en clair** (HTTP) : n'utilisez l'OTA que sur un réseau local de confiance, jamais en ouvrant le port 65280 sur Internet.
- Ne publiez pas un programme contenant vos identifiants WiFi (GitHub, forum...). Vous pouvez les placer dans un onglet `secrets.h` séparé.

9. Dépannage

Symptôme	Cause probable	Solution
'ArduinoOTA' was not declared in this scope	WiFiS3.h inclus après ArduinoOTA.h	Mettre <code>#include <WiFiS3.h></code> en premier.
undefined reference to 'InternalStorage'	Mauvaise carte sélectionnée ou bibliothèque trop ancienne	Sélectionner « Arduino UNO R4 WiFi » ; mettre à jour ArduinoOTA ($\geq 1.1.0$).
[OTA] Echec de connexion WiFi	SSID / mot de passe erronés, réseau 5 GHz, carte trop loin	Vérifier les identifiants, utiliser le 2,4 GHz, rapprocher la carte.
[OTA] Module WiFi absent !	Firmware ESP32-S3 corrompu ou carte défectueuse	Réinstaller le firmware avec <i>Outils</i> > <i>Firmware Updater</i> .
Pas de port réseau dans Outils > Port	mDNS bloqué (pare-feu, isolation WiFi « invités », VPN), IDE pas redémarré	Attendre 1 min, redémarrer l'IDE, autoriser l'IDE dans le pare-feu, désactiver le VPN ; sinon méthodes § 7.3 / 7.4.
Le port réseau apparaît mais l'envoi part en USB / erreur « missing ... »	platform.local.txt absent ou dans le mauvais dossier de version	Le recopier dans <code>renesas_uno/<version></code> utilisé, redémarrer l'IDE.
Unauthorized	Mot de passe saisi $\neq$ OTA_MDP	Ressaisir le bon mot de passe (l'IDE peut le mémoriser : le réinitialiser).
Error flashing the sketch après ~10 s	Délai de 10 s de l'outil arduinoOTA 1.3.0 trop court	Télécharger une version récente de arduinoOTA (github.com/arduino/arduinoOTA), remplacer l'exécutable et ajouter <code>-t 60</code> dans la ligne <code>upload.pattern</code> de <code>platform.local.txt</code> .
Internal Server Error / programme refusé	Programme trop gros ($> \sim 50\%$ de la flash)	Réduire la taille (bibliothèques, chaînes de caractères, tableaux constants).
L'envoi réussit mais le programme ne change pas	Moniteur série / réinitialisation pendant l'application	Fermer le moniteur série pendant l'envoi, ne pas couper l'alimentation.
Seul le premier envoi OTA fonctionne	Le programme envoyé ne contient pas les blocs OTA	Revenir à l'USB et téléverser un programme contenant les 4 blocs.
Envoi très lent ou échoue souvent	loop() bloquante, delay() longs, WiFi faible	Appeler <code>gererOTA()</code> souvent, supprimer les <code>delay()</code> longs, améliorer la réception.

Astuce : activez *Fichier* > *Préférences* > *Afficher la sortie détaillée pendant : Téléversement* pour voir la commande exacte lancée par l'IDE et le message d'erreur complet.

10. Alternative officielle : mise à jour par téléchargement

Le paquet UNO R4 fournit aussi la bibliothèque **OTAUpdate** (exemples *Fichier* > *Exemples* > *OTAUpdate*). Le principe est inverse : c'est la carte qui **télécharge** un fichier `.ota` depuis un serveur web, via l'ESP32-S3. C'est adapté au déploiement à distance (carte hors de votre réseau), mais demande de préparer et d'héberger les fichiers, et ne s'utilise pas directement depuis le bouton Téléverser de l'IDE. Pour un usage quotidien sur le réseau local, la méthode ArduinoOTA décrite dans ce manuel est plus simple.

Le service Arduino Cloud propose également l'OTA pour l'UNO R4 WiFi, via l'éditeur en ligne et un compte Arduino.

11. Récapitulatif rapide

#	Action	Quand
1	Installer la bibliothèque ArduinoOTA (J. Andrassy)	Une fois
2	Copier extras/renesas/platform.local.txt dans renesas_uno/<version>	Une fois + après chaque MAJ du paquet
3	Mettre à jour le firmware WiFi, redémarrer l'IDE	Une fois
4	Ajouter les blocs 1 et 2, activerOTA() et gererOTA()	Dans chaque programme
5	Premier téléversement en USB, noter l'IP	Une fois par carte
6	Outils > Port > port réseau, Téléverser, mot de passe	À chaque mise à jour

Annexe — Listing complet de OTA_R4_WiFi.ino

```

/* =====
 * OTA_R4_WiFi.ino - Mise à jour sans fil (OTA) pour Arduino UNO R4 WiFi
 * =====
 *
 * Ce fichier est un MODÈLE : il contient 4 blocs à recopier dans
 * n'importe quel programme pour lui ajouter le téléversement par WiFi
 * depuis l'IDE Arduino.
 *
 * BLOC 1 : includes + réglages          -> tout en haut du programme
 * BLOC 2 : fonctions OTA                -> avant setup()
 * BLOC 3 : 1 ligne dans setup()         -> activerOTA();
 * BLOC 4 : 1 ligne dans loop()          -> gererOTA();
 *
 * Option : modeOTA(duree) = « mode OTA dédié » bloquant (ex. sur bouton).
 *
 * Prérequis (voir le manuel PDF) :
 * - Bibliothèque « ArduinoOTA » de Juraj Andrassy (Gestionnaire de bibliothèques)
 * - Fichier platform.local.txt (extras/renesas de la bibliothèque) copié
 *   dans le dossier du paquet de cartes « renesas_uno »
 * - Premier téléversement par câble USB, les suivants par WiFi
 *
 * !!! IMPORTANT !!!
 * Chaque programme envoyé par OTA DOIT contenir ces blocs, sinon la carte
 * ne pourra plus être mise à jour par WiFi (retour obligatoire à l'USB).
 *
 * Limite : avec le stockage interne, la taille du programme est limitée
 * à la moitié de la flash, soit environ 128 Ko (< ~48 % affiché par l'IDE).
 * ===== */

/* #####
 * ## BLOC 1 – INCLUDES ET RÉGLAGES (à placer tout en haut du programme) ##
 * ##### */

#include <WiFiS3.h>          // Pilote WiFi de l'UNO R4 WiFi – TOUJOURS avant ArduinoOTA.h
#include <ArduinoOTA.h>      // Bibliothèque ArduinoOTA (J. Andrassy)

// --- Réglages à personnaliser -----
const char OTA_SSID[] = "NomDeMonReseau"; // Nom du réseau WiFi (2,4 GHz uniquement)
const char OTA_WIFIPW[] = "MotDePasseWiFi"; // Mot de passe du réseau WiFi
const char OTA_NOM[] = "UNO-R4-OTA"; // Nom de la carte affiché dans l'IDE (sans espace)
const char OTA_MDP[] = "password"; // Mot de passe demandé par l'IDE à chaque envoi OTA
// -> à changer ! (« password » = valeur par défaut)

#define OTA_LED_PIN 13 // LED témoin « L » (D13). Mettre -1 si D13 est utilisée ailleurs
#define OTA_WIFI_TIMEOUT 20000UL // Durée max d'une tentative de connexion WiFi (ms)
#define OTA_RECO_PERIODE 30000UL // Intervalle entre deux tentatives de reconnexion (ms)
#define OTA_DEBUG 1 // 1 = messages sur le moniteur série, 0 = silencieux

// --- Variables internes (ne pas modifier) -----
bool otaActif = false; // true quand le service OTA écoute
IPAddress otaIP; // Adresse IP utilisée par le service OTA
unsigned long otaDernierEssai = 0; // Horodatage de la dernière tentative WiFi

#if OTA_DEBUG
#define OTA_LOG(x) Serial.print(x)
#define OTA_LOGLN(x) Serial.println(x)
#else
#define OTA_LOG(x)
#define OTA_LOGLN(x)
#endif

/* #####
 * ## BLOC 2 – FONCTIONS OTA (à placer avant setup()) ##
 * ##### */

// Allume / éteint la LED témoin si elle est activée
void otaLed(bool etat) {
  #if OTA_LED_PIN >= 0

```

```

    digitalWrite(OTA_LED_PIN, etat ? HIGH : LOW);
#endif
}

// -----
// connecterWiFi() : se connecte au réseau, renvoie true si succès.
// Attention : WiFi.begin() est bloquant sur l'UNO R4 (plusieurs secondes).
// -----
bool connecterWiFi(unsigned long timeoutMs) {
    if (WiFi.status() == WL_NO_MODULE) {          // Le module ESP32-S3 ne répond pas
        OTA_LOGLN(F("[OTA] Module WiFi absent !"));
        return false;
    }

    OTA_LOG(F("[OTA] Connexion a ")); OTA_LOGLN(OTA_SSID);
    unsigned long t0 = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - t0 < timeoutMs) {
        WiFi.begin(OTA_SSID, OTA_WIFIPW);        // Tentative de connexion
        if (WiFi.status() != WL_CONNECTED) delay(1000);
    }
    if (WiFi.status() != WL_CONNECTED) {
        OTA_LOGLN(F("[OTA] Echec de connexion WiFi"));
        return false;
    }

    // Sur l'UNO R4, l'adresse IP peut rester à 0.0.0.0 quelques instants
    // après la connexion : on attend qu'elle soit attribuée par le DHCP.
    t0 = millis();
    while (WiFi.localIP() == IPAddress(0, 0, 0, 0) && millis() - t0 < 5000) delay(100);

    OTA_LOG(F("[OTA] Connecte, IP = ")); OTA_LOGLN(WiFi.localIP());
    return WiFi.localIP() != IPAddress(0, 0, 0, 0);
}

// -----
// Fonctions de rappel appelées par la bibliothèque pendant une mise à jour
// -----
void otaAuDebut() {                                // Un téléversement OTA commence
    OTA_LOGLN(F("[OTA] Reception d'un nouveau programme..."));
    otaLed(true);
    // >>> Ici : mettre les moteurs / sorties dangereuses en sécurité <<<
}

void otaAvantApplication() {                        // Programme reçu et vérifié, la carte va redémarrer
    OTA_LOGLN(F("[OTA] Ecriture du programme puis redémarrage"));
#ifdef OTA_DEBUG
    Serial.flush();                                // Vider le tampon série avant le redémarrage
#endif
}

// -----
// activerOTA() : LE « VOID » PRINCIPAL – passe la carte en mode OTA.
// Connecte le WiFi si besoin, puis démarre le service qui écoute l'IDE
// (port TCP 65280 + annonce mDNS pour apparaître dans Outils > Port).
// Peut être appelé plusieurs fois sans risque.
// -----
void activerOTA() {
    if (otaActif) return;                          // Déjà actif : rien à faire

#ifdef OTA_LED_PIN >= 0
    pinMode(OTA_LED_PIN, OUTPUT);
#endif

    // Vérification (facultative) du firmware du module WiFi ESP32-S3
    String fw = WiFi.firmwareVersion();
    if (fw < WIFI_FIRMWARE_LATEST_VERSION) {
        OTA_LOG(F("[OTA] Firmware WiFi ")); OTA_LOG(fw);
        OTA_LOGLN(F(" : mise à jour conseillée (Outils > Firmware Updater)"));
    }
}

```

```

    if (WiFi.status() != WL_CONNECTED && !connecterWiFi(OTA_WIFI_TIMEOUT)) {
        otaDernierEssai = millis(); // gererOTA() réessaiera plus tard
        return;
    }

    ArduinoOTA.onStart(otaAuDebut); // Rappel : début de réception
    ArduinoOTA.beforeApply(otaAvantApplication); // Rappel : juste avant redémarrage

    otaIP = WiFi.localIP();
    ArduinoOTA.begin(otaIP, OTA_NOM, OTA_MDP, InternalStorage); // Démarrage du service
    otaActif = true;

    OTA_LOG(F("[OTA] Pret ! Port reseau : ")); OTA_LOG(OTA_NOM);
    OTA_LOG(F(" at ")); OTA_LOGLN(otaIP);
}

// -----
// gererOTA() : à appeler à CHAQUE tour de loop().
// - écoute les demandes de téléversement de l'IDE
// - relance le WiFi et le service si la connexion a été perdue
// Ne bloque pas (sauf pendant une reconnexion WiFi, toutes les 30 s max).
// -----
void gererOTA() {
    // 1) WiFi perdu : on coupe le service et on retente périodiquement
    if (WiFi.status() != WL_CONNECTED) {
        if (otaActif) {
            OTA_LOGLN(F("[OTA] WiFi perdu"));
            ArduinoOTA.end(); // Arrêt propre du service
            otaActif = false;
        }
        if (millis() - otaDernierEssai >= OTA_RECO_PERIODE) {
            otaDernierEssai = millis();
            activerOTA(); // Reconnexion + redémarrage du service
        }
        return;
    }

    // 2) WiFi présent mais service pas encore démarré (ex. échec au setup)
    if (!otaActif) {
        if (millis() - otaDernierEssai >= OTA_RECO_PERIODE) {
            otaDernierEssai = millis();
            activerOTA();
        }
        return;
    }

    // 3) L'adresse IP a changé (bail DHCP renouvelé) : on relance le service
    if (WiFi.localIP() != otaIP) {
        ArduinoOTA.end();
        otaActif = false;
        activerOTA();
        return;
    }

    // 4) Cas normal : traiter une éventuelle demande de l'IDE
    ArduinoOTA.poll();
}

// -----
// modeOTA(dureeMs) : « mode OTA dédié », BLOQUANT.
// Le programme principal est suspendu, la LED clignote, la carte attend
// uniquement un téléversement. dureeMs = 0 -> attente infinie.
// Utile si l'on ne veut PAS laisser l'OTA actif en permanence
// (ex. déclenché par un bouton au démarrage).
// -----
void modeOTA(unsigned long dureeMs) {
    activerOTA();
    OTA_LOGLN(F("[OTA] Mode OTA dedie : en attente d'un televersement..."));

    unsigned long t0 = millis();

```

```

while (dureeMs == 0 || millis() - t0 < dureeMs) {
  gererOTA(); // Si un envoi arrive, la carte redémarre ici
  otaLed((millis() / 250) % 2); // Clignotement rapide = mode OTA
  delay(5);
}
otaLed(false);
OTA_LOGLN(F("[OTA] Fin du mode OTA dedie, reprise du programme"));
}

/* #####
 * ## EXEMPLE DE PROGRAMME QUELCONQUE UTILISANT LES BLOCS CI-DESSUS ##
 * ##### */

#define BOUTON_OTA 2 // Exemple : bouton entre D2 et GND (facultatif)

void setup() {
  Serial.begin(115200);
  unsigned long t = millis();
  while (!Serial && millis() - t < 3000); // Attend le moniteur série 3 s max

  // ---- BLOC 3 : UNE ligne dans setup() -----
  activerOTA(); // OTA toujours disponible pendant le fonctionnement
  // -----

  // Variante « à la demande » (à la place de activerOTA()) :
  // pinMode(BOUTON_OTA, INPUT_PULLUP);
  // if (digitalRead(BOUTON_OTA) == LOW) modeOTA(120000UL); // 2 min d'attente

  // ... votre setup() habituel ...
}

void loop() {
  // ---- BLOC 4 : UNE ligne au début de loop() -----
  gererOTA();
  // -----

  // ... votre loop() habituel ...
  // Évitez les delay() longs : pendant un delay(), l'OTA n'est pas écouté.
  // Exemple non bloquant : un message toutes les 5 s.
  static unsigned long dernier = 0;
  if (millis() - dernier >= 5000) {
    dernier = millis();
    Serial.println(F("Programme en cours... (version 1)"));
  }
}

```